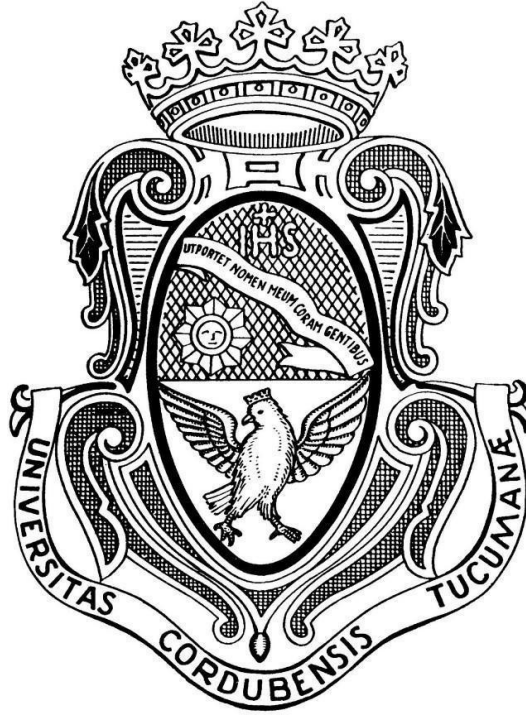


Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Sistemas de Computación

Trabajo Práctico N° 4 - Módulos de Kernel y Llamadas a Sistema

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

1 de junio de 2026

Índice

1	Desafío #1	2
1.1	¿Qué es checkinstall y para qué sirve?	2
1.2	Usar checkinstall para empaquetar un hello world	2
1.3	Seguridad del Kernel	4
2	Desafío #2	4
2.1	¿Qué funciones tiene disponible un programa y un módulo?	4
2.2	Espacio de Usuario y Espacio de Kernel	4
2.3	Espacio de Datos	4
3	Drivers y Contenido de /dev	5
4	Módulos de Kernel	6
4.1	Creación y Análisis de un Módulo de Kernel	6
4.2	Preguntas	8
5	Link al Repositorio	18

1. Desafío #1

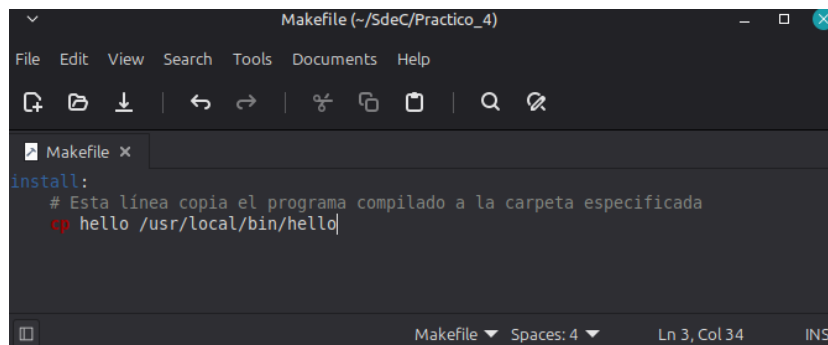
1.1. ¿Qué es checkinstall y para qué sirve?

Checkinstall es una herramienta que actúa como intermediario durante la instalación de un programa compilado desde su código fuente. Se utiliza en lugar de `make install`, y permite “anotar” cada archivo que se está copiando en el disco, creando e instalando automáticamente un paquete para la distribución que se esté usando. Una de las ventajas que tiene usar checkinstall es que luego es posible borrar todos los archivos de manera más sencilla en lugar de ir uno por uno. Además, ahora sistema operativo tiene un registro de que el software existe, lo cual evita conflictos con otros programas y mantiene el sistema más ordenado.

1.2. Usar checkinstall para empaquetar un hello world

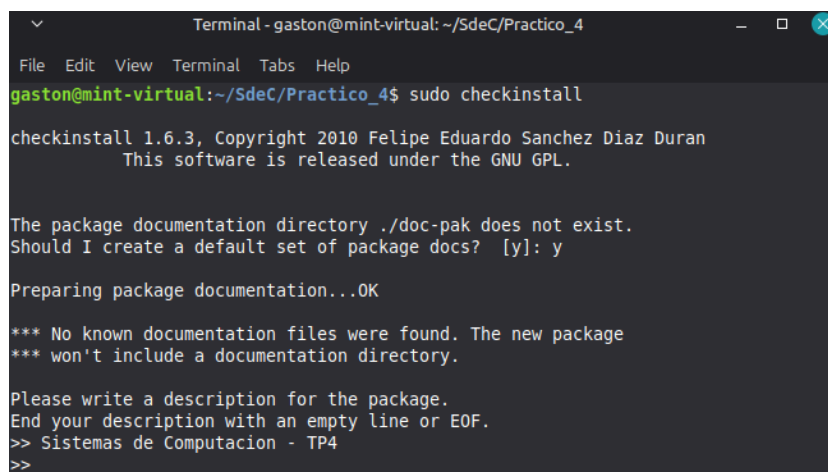
Para que checkinstall funcione y genere el paquete de instalación, necesita que el proyecto tenga un sistema de construcción, que típicamente es un archivo Makefile. Entonces, este archivo se encarga de indicarle al sistema operativo cómo compilar el programa, y a qué carpeta del sistema debe copiarlo.

En Linux, el Filesystem Hierarchy Standard (FHS) establece que los programas ejecutables (binarios) que el usuario compila e instala por su cuenta deben ir en “`/usr/local/bin/`”.



```
Makefile (~/SdeC/Practico_4)
File Edit View Search Tools Documents Help
[Icons]
Makefile x
install:
# Esta línea copia el programa compilado a la carpeta especificada
cp hello /usr/local/bin/hello
[Icons] Makefile Spaces: 4 Ln 3, Col 34 INS
```

Figura 1: Archivo Makefile.



```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_4
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_4$ sudo checkinstall

checkinstall 1.6.3, Copyright 2010 Felipe Eduardo Sanchez Diaz Duran
This software is released under the GNU GPL.

The package documentation directory ./doc-pak does not exist.
Should I create a default set of package docs? [y]: y

Preparing package documentation...OK

*** No known documentation files were found. The new package
*** won't include a documentation directory.

Please write a description for the package.
End your description with an empty line or EOF.
>> Sistemas de Computacion - TP4
>>
```

Figura 2: Comando checkinstall.

```
*****
**** Debian package creation selected ****
*****

*** Warning: The package name "Practico_4" contains upper case
*** Warning: letters. dpkg might not like that so I changed
*** Warning: them to lower case.

*** Warning: The package name "practico_4" contains illegal
*** Warning: characters. dpkg might not like that so I changed
*** Warning: them to dashes.

This package will be built according to these values:

0 - Maintainer: [ root@mint-virtual ]
1 - Summary: [ Sistemas de Computacion - TP4 ]
2 - Name: [ practico-4 ]
3 - Version: [ 20260529 ]
4 - Release: [ 1 ]
5 - License: [ GPL ]
6 - Group: [ checkinstall ]
7 - Architecture: [ amd64 ]
8 - Source location: [ Practico_4 ]
9 - Alternate source location: [ ]
10 - Requires: [ ]
11 - Recommends: [ ]
12 - Suggests: [ ]
13 - Provides: [ practico-4 ]
14 - Conflicts: [ ]
15 - Replaces: [ ]
16 - Prerequires: [ ]

Enter a number to change any of them or press ENTER to continue: 
```

Figura 3: Detalles de la creación del paquete.

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_4
File Edit View Terminal Tabs Help

Installing with make install...

===== Installation results =====
# Esta línea copia el programa compilado a la carpeta especificada
cp hello /usr/local/bin/hello

===== Installation successful =====

Copying files to the temporary directory...OK
Stripping ELF binaries and libraries...OK
Compressing man pages...OK
Building file list...OK
Building Debian package...OK
Installing Debian package...OK
Erasing temporary files...OK
Writing backup package...OK
Deleting temp dir...OK

*****

Done. The new package has been installed and saved to
/home/gaston/SdeC/Practico_4/practico-4_20260529-1_amd64.deb

You can remove it from your system anytime using:

    dpkg -r practico-4

*****

gaston@mint-virtual:~/SdeC/Practico_4$
```

Figura 4: Instalación Exitosa.

El paquete creado e instalado es de tipo .deb porque se utiliza Linux Mint, que está basado en Ubuntu, el cual está construido sobre Debian.

A su vez se comprueba que existe un comando para borrar el archivo:

```
dpkg -r practico-4
```

1.3. Seguridad del Kernel

Un rootkit es un tipo de software malicioso diseñado para darle a un atacante acceso de administrador (root) a un equipo mientras se oculta a sí mismo. El rootkit de Kernel, es aquel en el cual el atacante carga su propio módulo de Kernel en el sistema operativo de un equipo, obteniendo control sobre el hardware y software. En caso de tener éxito, el atacante puede esconder sus archivos para que el comando “ls” no los vea, puede esconder procesos maliciosos para que no aparezcan en el administrador de tareas y puede desactivar funciones del firewall o antivirus.

Para evitar estos problemas existen las siguientes soluciones:

Firma de Módulos: Históricamente, con permisos de administrador (sudo) se podía cargar cualquier módulo al kernel usando comandos como insmod o modprobe. Si un atacante conseguía la contraseña de usuario, podía cargar su rootkit. Es por ello que se introdujeron los módulos firmados criptográficamente, en los que el desarrollador del módulo utiliza una clave privada para estampar una firma digital en el archivo .ko. A su vez, el sistema operativo tiene guardada una lista de claves públicas confiables, entonces cuando alguien intenta cargar un módulo en el kernel, se verifica que la firma se haya hecho con una clave privada que coincide con una clave pública autorizada.

Secure Boot: Cuando se habilita el Secure Boot desde la BIOS, el kernel se niega a ejecutar cualquier código a nivel de sistema que no esté firmado de forma oficial.

Entonces, si se desea desarrollar un módulo de kernel propio, es necesario crear un par de claves (privada y pública), firmar el módulo utilizando la clave privada (mediante una herramienta llamada sign-file) e inscribir la clave pública en la lista de confianza de la placa madre de la computadora (usando una herramienta llamada Machine Owner Key).

2. Desafío #2

2.1. ¿Qué funciones tiene disponible un programa y un módulo?

Programa: Un programa normal de aplicación de usuario se ejecuta en el espacio de usuario. Tiene a su disposición bibliotecas de usuario (por ejemplo la biblioteca estándar de C libc), por lo que puede usar funciones como printf() para imprimir en pantalla o malloc() para reservar memoria.

Módulo de Kernel: Es un fragmento de código que se carga dinámicamente dentro del kernel del sistema operativo. No tiene acceso a ninguna biblioteca de usuario, sino que solo puede usar funciones que el kernel exporta (Kernel API). De esta forma, en lugar de utilizar printf() se usa printk(), y en lugar de utilizar malloc() se usa kcalloc().

2.2. Espacio de Usuario y Espacio de Kernel

Espacio de Usuario: Es un área de privilegios restringidos donde se ejecutan la mayoría de las aplicaciones. En este espacio, el software no puede acceder directamente al hardware ni a la memoria de otros programas, por cuestiones de seguridad. Si un programa falla, se cierra pero el sistema operativo sigue funcionando sin problemas.

Espacio de Kernel: Es el área de mayor privilegio, donde se ejecuta el núcleo del sistema operativo y sus módulos. A diferencia del espacio de usuario, acá se tiene acceso irrestricto a toda la memoria del sistema y se controla directamente el hardware, por lo que un fallo en un código que se ejecuta en este espacio es crítico y suele provocar que todo el sistema colapse.

Cabe destacar que cuando un programa en el espacio de usuario necesita hacer algo que requiere privilegios altos (como leer un archivo de disco), debe pedirle permiso al kernel, y esto se logra mediante una System Call.

2.3. Espacio de Datos

En Programas de Usuario: La gestión de memoria es manejada por el sistema operativo de forma virtual. Un programa tiene una pila que puede crecer dinámicamente y los desarrolladores pueden crear estructuras de datos enormes sin que se desborde la pila.

En el Kernel: En este caso el espacio es escaso y crítico. La pila (kernel stack) es extremadamente pequeña (alrededor de 4 u 8 KB por hilo de ejecución), por lo que al programar módulos de kernel la memoria debe ser asignada dinámicamente. Además, la memoria del kernel generalmente no se puede paginar, por lo que utiliza RAM física.

3. Drivers y Contenido de /dev

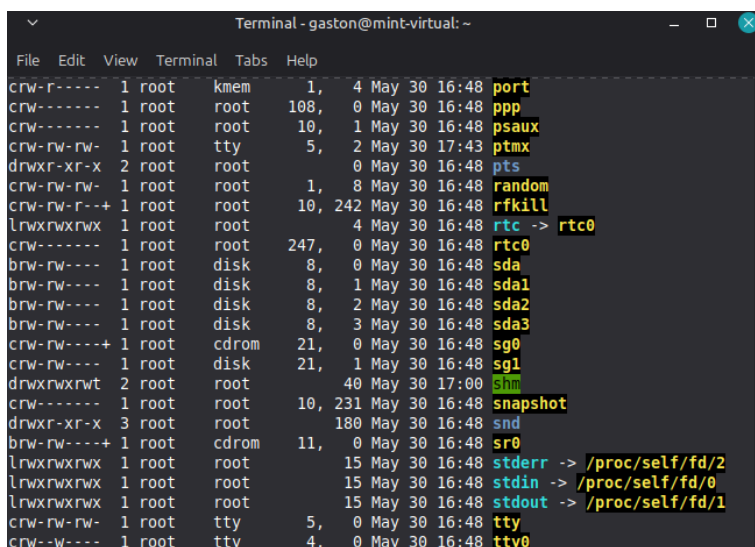
Drivers: Son piezas de software (casi siempre implementadas como módulos del kernel) que permiten que el sistema operativo y los programas comprendan cómo interactuar con un hardware específico (por ejemplo un teclado o un disco duro), abstrayendo de la complejidad electrónica y ofreciendo una interfaz de software estándar.

Existen 3 tipos fundamentales de drivers:

- Drivers de Caracteres: Utilizan la letra c, y manejan el hardware byte a byte, de forma secuencial y sin usar un buffer de memoria intermedio (por ejemplo /dev/null).
- Drivers de Bloque: Utilizan la letra b, y manejan el hardware leyendo y escribiendo bloques de datos de tamaño fijo. A su vez, permiten el acceso aleatorio (ir a cualquier parte del disco) y utilizan la memoria caché del sistema (por ejemplo /dev/sda).
- Drivers de Red: Los drivers de red no tienen un archivo en /dev (por lo que no les corresponde ninguna letra), sino que como las redes están basadas en paquetes y protocolos complejos, se interactúa con ellos mediante sockets y herramientas de red y no mediante operaciones de lectura/escritura de archivos tradicionales.

El Directorio /dev: Este directorio contiene archivos especiales llamados nodos de dispositivo, los cuales no contienen datos normales sino que se utilizan para los programas de usuario puedan comunicarse con los drivers en el kernel. La ventaja es que se simplifica mucho la programación porque se pueden usar las mismas System Calls para diversos dispositivos.

Si se usa el comando “ls -l /dev” se puede observar que la primera letra que aparece indica el tipo de archivo, y al lado del nombre del dispositivo hay 2 números separados por una coma, donde el número de la izquierda representa qué driver es y el de la derecha representa cuál dispositivo es de dicho driver.



```

Terminal - gaston@mint-virtual: ~
File Edit View Terminal Tabs Help
crw-r----- 1 root kmem 1, 4 May 30 16:48 port
crw-r----- 1 root root 108, 0 May 30 16:48 ppp
crw-r----- 1 root root 10, 1 May 30 16:48 psaux
crw-rw-rw- 1 root tty 5, 2 May 30 17:43 ptmx
dwxr-xr-x 2 root pts 0 May 30 16:48 pts
crw-rw-rw- 1 root random 1, 8 May 30 16:48 random
crw-rw-r--+ 1 root rfkill 10, 242 May 30 16:48 rfkill
lrwxrwxrwx 1 root rtc -> rtc0
crw-r----- 1 root rtc0 247, 0 May 30 16:48 rtc0
brw-rw---- 1 root sda 8, 0 May 30 16:48 sda
brw-rw---- 1 root sda1 8, 1 May 30 16:48 sda1
brw-rw---- 1 root sda2 8, 2 May 30 16:48 sda2
brw-rw---- 1 root sda3 8, 3 May 30 16:48 sda3
crw-rw-rw-+ 1 root sg0 21, 0 May 30 16:48 sg0
crw-rw-rw- 1 root sg1 21, 1 May 30 16:48 sg1
drwxrwxrwt 2 root shm 40 May 30 17:00 shm
crw-r----- 1 root snapshot 10, 231 May 30 16:48 snapshot
dwxr-xr-x 3 root snd 180 May 30 16:48 snd
brw-rw-rw-+ 1 root sr0 11, 0 May 30 16:48 sr0
lrwxrwxrwx 1 root stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty 5, 0 May 30 16:48 tty
crw--w---- 1 root tty0 4, 0 May 30 16:48 tty0
    
```

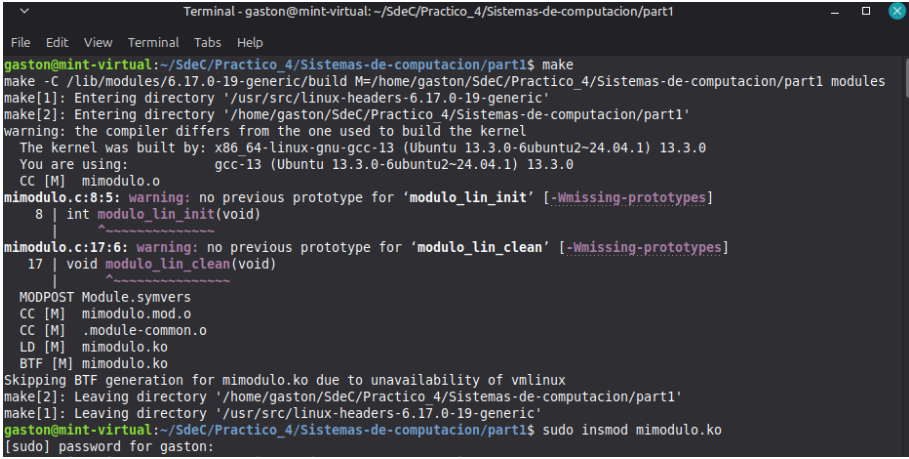
Figura 5: Comando “ls -l /dev”.

4. Módulos de Kernel

4.1. Creación y Análisis de un Módulo de Kernel

Se ejecutan los siguientes pasos:

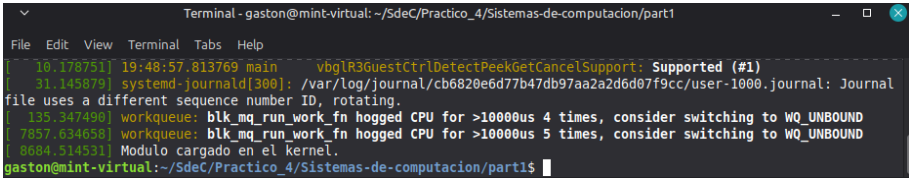
```
cd part1
make
sudo insmod mimodulo.ko
sudo dmesg
lsmod | grep mod
```



```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_4/Sistemas-de-computacion/part1
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ make
make -C /lib/modules/6.17.0-19-generic/build M=/home/gaston/SdeC/Practico_4/Sistemas-de-computacion/part1 modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-19-generic'
make[2]: Entering directory '/home/gaston/SdeC/Practico_4/Sistemas-de-computacion/part1'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M] mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
8 | int modulo_lin_init(void)
  | ^~~~~~
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
17 | void modulo_lin_clean(void)
   | ^~~~~~
MODPOST Module.symvers
CC [M] mimodulo.mod.o
CC [M] .module-common.o
LD [M] mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/gaston/SdeC/Practico_4/Sistemas-de-computacion/part1'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-19-generic'
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ sudo insmod mimodulo.ko
[sudo] password for gaston:
```

Figura 6: Comandos make e insmod.

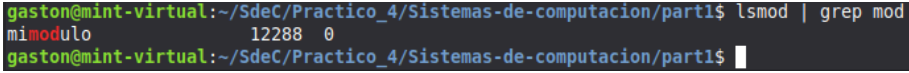
El comando make se encarga de crear el archivo mimodulo.ko, mientras que el comando insmod se encarga de insertar el módulo dentro del kernel.



```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_4/Sistemas-de-computacion/part1
File Edit View Terminal Tabs Help
10.178751] 19:48:57.813769 main vbglR3GuestCtrlDetectPeekGetCancelSupport: Supported (#1)
31.145879] systemd-journald[300]: /var/log/journal/cb6820e6d77b47db97aa2a2d6d07f9cc/user-1000.journal: Journal
file uses a different sequence number ID, rotating.
135.347498] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 4 times, consider switching to WQ_UNBOUND
7857.634658] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 5 times, consider switching to WQ_UNBOUND
8684.514531] Modulo cargado en el kernel.
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

Figura 7: Comando dmesg.

Mediante el comando dmesg se puede verificar la correcta carga del módulo.



```
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ lsmod | grep mod
mimodulo                12288  0
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

Figura 8: Comando lsmod — grep.

Con estos comandos se realiza el filtro de los módulos de kernel para mostrar solamente los que contengan “mod”, de forma que se vea el módulo previamente creado y cargado.

Ahora se realizan los siguientes pasos para comprobar que se puede remover el módulo:

```
sudo rmmod mimodulo
sudo dmesg
lsmod | grep mod
cat /proc/modules | grep mod
```

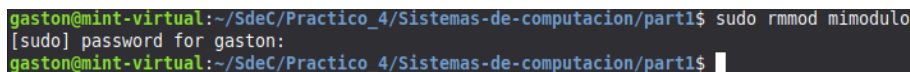


Figura 9: Comando rmmod.

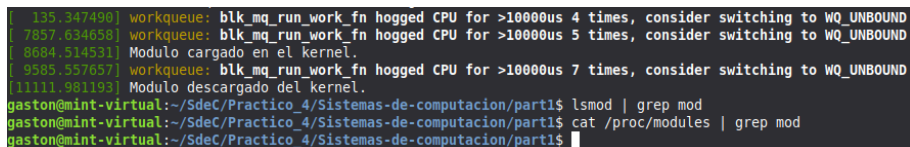


Figura 10: Comandos lsmod y cat.

Ambos comandos se utilizan para verificar la correcta remoción del módulo.
Luego se observa la información del módulo:

```
modinfo mimodulo.ko
modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko
```

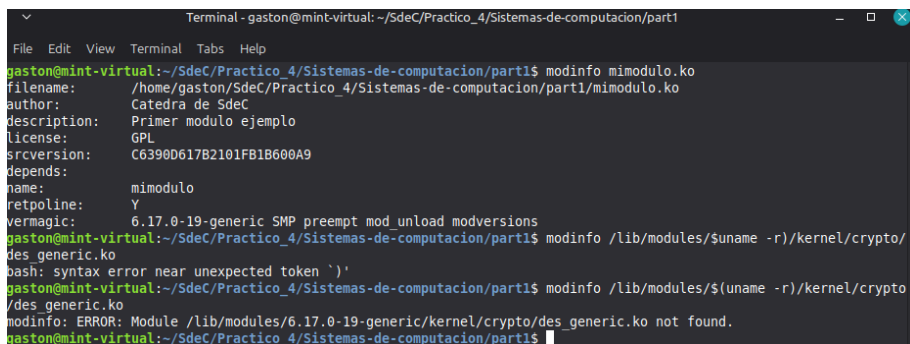


Figura 11: Comandos modinfo.

Como se puede observar, en esta versión de linux no se encuentra disponible en esta ruta el módulo recomendado, por lo que en su lugar se ejecuta el comando modinfo para el módulo aesni_intel, según la siguiente figura.


```
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ modinfo aesni_intel
filename:           /lib/modules/6.17.0-19-generic/kernel/arch/x86/crypto/aesni-intel.ko.zst
alias:              crypto-aes
alias:              aes
license:            GPL
description:        AES cipher and modes, optimized with AES-NI or VAES instructions
srcversion:         495482AA6AB854C21823B78
alias:              cpu:type:x86,ven*fam*mod*:feature:*0099*
depends:
intree:            Y
name:              aesni_intel
retpoline:         Y
vermagic:          6.17.0-19-generic SMP preempt mod_unload modversions
sig_id:            PKCS#7
signer:            Build time autogenerated kernel key
sig_key:           16:05:49:1C:C7:B1:9E:30:CA:07:06:DD:8C:6A:29:46:EF:F1:37:D0
sig_hashalgo:      sha512
signature:         99:12:5C:21:0B:BA:75:BD:FA:2C:35:D2:29:3C:86:67:26:8C:D3:A0:
CE:66:39:BF:A0:EA:0E:21:59:16:28:91:FF:F6:0E:E3:1A:F0:F6:9B:
70:1C:DF:F6:1C:02:DA:D3:9D:B6:40:B5:12:9A:19:D3:17:37:5C:BA:
47:03:32:99:F3:B5:1E:5F:C2:7D:EA:3A:3E:5C:94:1F:C1:78:EE:59:
B0:1D:5B:99:2D:20:54:98:7B:B8:74:2D:51:BD:C2:4E:F1:C4:C9:1F:
49:1F:63:69:5D:A2:2B:C3:4C:F6:DF:53:CD:A1:32:7B:F4:C9:E6:FF:
C6:E8:9F:89:2D:56:9E:80:7A:2F:D9:2E:43:C8:B0:3F:72:81:7C:4D:
29:96:EE:06:2F:1B:18:92:1B:97:FD:C8:16:D8:38:6E:89:53:68:E3:
66:39:F9:35:D8:B6:D4:71:6A:9B:61:45:15:71:39:6D:61:75:C3:29:
8A:E2:41:45:68:D4:9F:B4:BF:E8:46:EC:7B:6C:85:31:73:0F:1C:ED:
00:0E:94:50:DF:87:9C:B1:25:C7:9D:AD:41:02:D1:42:94:A8:9F:1A:
9E:ED:A3:AA:3C:25:25:33:BB:B2:DD:08:88:10:81:41:07:2A:48:95:
CB:5B:F5:A8:0F:F1:76:0E:9F:4B:AF:A3:B2:04:D1:ED:44:87:6D:43:
35:1E:B3:00:EC:44:C0:31:6D:D0:04:8A:47:73:99:C4:C8:17:22:48:
B8:FC:48:D8:C5:E9:58:78:65:BB:30:D5:34:7A:43:6F:42:99:29:F7:
09:0C:68:DC:F7:E8:3A:3E:67:07:36:E7:CE:52:D7:A4:AC:79:EC:E2:
04:41:4D:FD:A0:06:F7:CE:F6:A7:6E:7B:52:A5:0D:0C:46:A7:E0:A0:
1F:4A:68:46:84:F3:1B:13:F5:7A:B6:0B:55:93:34:76:2F:2D:06:F1:
6B:98:E9:B9:97:E5:04:4C:BD:1F:9A:DA:C5:65:AE:21:3C:F4:CC:14:
31:B6:83:00:9B:92:58:39:A2:E7:31:3E:06:8C:43:83:E9:E6:82:EB:
9A:69:77:D3:9D:B4:79:74:DC:3A:1A:AC:E1:42:54:21:96:C9:04:5F:
EE:D9:E2:CA:99:C9:1A:A2:B8:EC:0F:4B:00:1C:93:A3:75:07:46:EC:
58:09:21:FB:1C:59:13:86:57:53:96:8E:2C:E8:2E:A2:08:F3:19:E6:
0C:D1:A2:5F:5D:25:75:DB:7C:35:68:0C:B8:58:21:8D:E5:32:59:9E:
4E:D7:2E:4D:28:ED:6C:14:6C:BF:27:C9:1E:EC:14:74:13:34:A0:6C:
D5:96:9D:59:52:B9:79:7D:03:F5:6B:F5
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

Figura 12: Comando modinfo para el módulo aesni_intel.

4.2. Preguntas

1. ¿Qué diferencias se pueden observar entre los dos modinfo?

Respuesta 1: En el módulo oficial aparece intree (que significa que fue compilado de forma oficial), sig_id (es el formato de la firma), signer (quién lo firmó), sig_key (indica que se utilizó una firma privada), sig_hashalgo (algoritmo de hash utilizado) y signature (la firma criptográfica, que garantiza la autenticidad).

2. ¿Qué drivers/módulos están cargados en mi pc?

Respuesta 2: Se puede utilizar el comando lsmod para imprimir todos los módulos cargados en la pc:

```
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ lsmod
Module                  Size  Used by
snd_seq_dummy           12288  0
snd_hrtimer             12288  1
qrtr                   53248  2
snd_intel8x0            53248  1
snd_ac97_codec          196608 1 snd_intel8x0
ac97_bus                12288  1 snd_ac97_codec
intel_rapl_msr          20480  0
snd_pcm                 200704 2 snd_intel8x0,snd_ac97_codec
intel_rapl_common       53248  1 intel_rapl_msr
snd_seq_midi            24576  0
snd_seq_midi_event      16384  1 snd_seq_midi
intel_uncore_frequency_common 20480  0
snd_rawmidi             57344  1 snd_seq_midi
binfmt_misc            24576  1
intel_pmc_core          135168  0
pmt_telemetry           16384  1 intel_pmc_core
pmt_discovery           20480  1 pmt_telemetry
pmt_class               20480  2 pmt_telemetry,pmt_discovery
snd_seq                 122880 9 snd_seq_midi,snd_seq_midi_event,snd_seq_dummy
intel_pmc_ssram_telemetry 16384  1 intel_pmc_core
intel_vsec              24576  2 intel_pmc_ssram_telemetry,pmt_telemetry
snd_seq_device          16384  3 snd_seq,snd_seq_midi,snd_rawmidi
snd_timer               53248  3 snd_seq,snd_hrtimer,snd_pcm
nls_iso8859_1           12288  1
i2c_piix4               32768  0
rapl                    20480  0
snd                     143360 11 snd_seq,snd_seq_device,snd_intel8x0,snd_timer,snd_ac97_codec,snd_pcm,snd_rawmidi
vboxguest              507904  6
i2c_smbus               20480  1 i2c_piix4
soundcore               16384  1 snd
input_leds              12288  0
joydev                  32768  0
serio_raw               20480  0
mac_hid                 12288  0
sch_fq_codel            24576  2
vmwgfx                  471040  2
```

Figura 13: Comando lsmod parte 1.

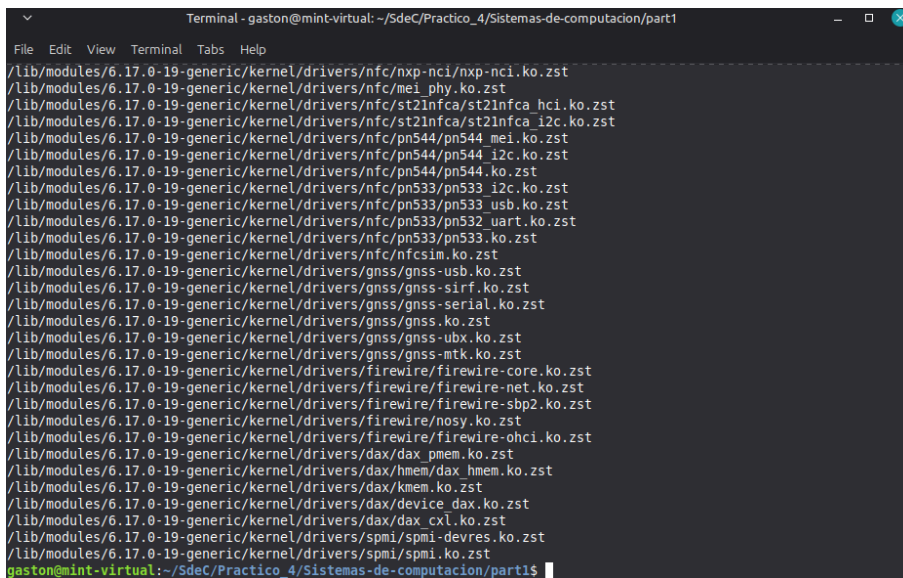
```
vmwgfx                  471040  2
drm_ttm_helper          16384  2 vmwgfx
ttm                     126976 2 vmwgfx,drm_ttm_helper
msr                     12288  0
parport_pc              53248  0
ppdev                   24576  0
lp                      32768  0
parport                 73728  3 parport_pc,lp,ppdev
efi_pstore              12288  0
nfnetlink               20480  1
dmi_sysfs               20480  0
lp_tables               32768  0
x_tables                65536  1 lp_tables
autofs4                 57344  2
btrfs                  2150400 0
blake2b_generic         20480  0
raid10                  73728  0
raid456                 192512 0
async_raid6_recov       20480  1 raid456
async_memcpy            16384  2 raid456,async_raid6_recov
async_pq                20480  2 raid456,async_raid6_recov
async_xor               16384  3 async_pq,raid456,async_raid6_recov
async_tx                16384  5 async_pq,async_memcpy,async_xor,raid456,async_raid6_recov
xor                     24576  2 async_xor,btrfs
raid6_pq                131072 4 async_pq,btrfs,raid456,async_raid6_recov
raid1                   61440  0
raid0                   24576  0
linear                  16384  0
dm_mirror               28672  0
dm_region_hash          24576  1 dm_mirror
dm_log                  20480  2 dm_region_hash,dm_mirror
hid_generic             12288  0
usbhid                  77024  0
hid                     262144 2 usbhid,hid_generic
ahci                     49152  2
vga16fb                 32768  0
polyval_clmulni         12288  0
libahci                 53248  1 ahci
e1000                   180224 0
video                   77824  0
psmouse                 217088 0
ghash_clmulni_intel     12288  0
vgastate                20480  1 vga16fb
pata_acpi               12288  0
wmi                     28672  1 video
aesni_intel             98304  0
```

Figura 14: Comando lsmod parte 2.

3. ¿Cuáles no están cargados pero están disponibles? ¿Qué pasa cuando el driver de un dispositivo no está disponible?

Respuesta 3: Mediante el siguiente comando `find` se pueden ver todos los módulos disponibles, mientras que en las figuras 13 y 14 se ven los cargados, entonces la diferencia son los que no están cargados pero sí están disponibles.

```
find /lib/modules/$(uname -r) -type f -name '*.ko*'
```



```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_4/Sistemas-de-computacion/part1
File Edit View Terminal Tabs Help
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/nxp-nci/nxp-nci.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/mei_phy.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/st2lnfca/st2lnfca_hci.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/st2lnfca/st2lnfca_i2c.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn544/pn544_mei.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn544/pn544_i2c.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn544/pn544.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn533/pn533_i2c.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn533/pn533_usb.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn533/pn532_uart.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/pn533/pn533.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/nfc/nfcsim.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss-usb.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss-sirf.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss-serial.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss-ubx.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/gnss/gnss-mtk.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/firewire/firewire-core.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/firewire/firewire-net.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/firewire/firewire-sbp2.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/firewire/nosy.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/firewire/firewire-ohci.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/dax/dax_pmem.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/dax/hmem/dax_hmem.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/dax/kmem.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/dax/device_dax.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/dax/dax_cxl.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/spmi/spmi-devres.ko.zst
/lib/modules/6.17.0-19-generic/kernel/drivers/spmi/spmi.ko.zst
gaston@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

Figura 15: Comando `find` para ver los módulos disponibles.

Si el driver de un dispositivo no está disponible, entonces el sistema operativo no sabe cómo comunicarse con el hardware, por lo tanto el dispositivo no va a funcionar o va a funcionar de manera genérica y limitada.

4. Correr `hwinfo` en una pc real con hw real y agregar la url de la información de hw en el reporte.

Respuesta 4: En este caso no se cuenta con la posibilidad de correr el comando en hardware real porque solamente se cuenta con el entorno linux mediante VirtualBox.

5. ¿Qué diferencia existe entre un módulo y un programa?

Respuesta 5: Existen las siguientes diferencias principales:

- Un programa tiene un `main`, donde ejecuta una secuencia de instrucciones para hacer una tarea determinada y cuando termina, el sistema operativo limpia sus recursos. Un módulo posee las funciones `module_init()` y `module_exit()`, que se utilizan para registrar la funcionalidad del módulo en el kernel, por lo que luego se queda esperando a que se requiera usar dicha funcionalidad.
- No tienen acceso a las mismas funciones, ya que los programas pueden usar las bibliotecas de usuario mientras que los módulos pueden usar la API de kernel.
- Tienen distintos espacios de ejecución, ya que los programas cuentan con el espacio de usuario mientras que los módulos cuentan con el espacio de kernel. Estos espacios tienen diferentes niveles de privilegios.
- Tienen diferentes manejos de fallos, ya que ante el fallo de un programa simplemente se cancela la ejecución del programa, mientras que si falla un módulo de kernel el sistema operativo se detiene por completo.

6. ¿Cómo se puede ver una lista de las llamadas al sistema que realiza un simple `helloworld` en c?

Respuesta 6: Se puede hacer mediante el comando `strace`. En este caso se ejecuta sobre el archivo ejecutable del código `hello.c`, utilizado previamente.

```
Terminal - ganton@mint-virtual: ~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

```
File Edit View Terminal Tabs Help

ganton@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$ strace ./hello
execve("./hello", ["/.hello"], 0x7ffe47d2c5b0 /* 53 vars */) = 0
brk(NULL)                               = 0x64907cf7f000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x795ef373d000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=79023, ...}) = 0
mmap(NULL, 79023, PROT_READ, MAP_PRIVATE, 3, 0) = 0x795ef3729000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\0\0\0\220\243\2\0\0\0\0\0"... , 832) = 832
pread64(3, {"\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0... ", 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, {"\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0... ", 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x795ef3400000
mmap(0x795ef3428000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x795ef3428000
mmap(0x795ef35b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x795ef35b0000
mmap(0x795ef35ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xfife000) = 0x795ef35ff000
mmap(0x795ef3605000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x795ef3605000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x795ef3726000
arch_prctl(ARCH_SET_FS, 0x795ef3726740) = 0
set_tid_address(0x795ef3726a10)         = 10129
set_robust_list(0x795ef3726a20, 24)     = 0
rseq(0x795ef3727000, 0x20, 0, 0x53053053) = 0
mprotect(0x795ef35ff000, 16384, PROT_READ) = 0
mprotect(0x6490703af000, 4096, PROT_READ) = 0
mprotect(0x795ef377d000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x795ef3729000, 79023)           = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
getrandom("\xae2\xa2\x3d\x86\x36\xb8\b\x81\x46", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x64907cf7f000
brk(0x64907c818000)                     = 0x64907c818000
write(1, "\n", 1)                        = 1
write(1, " Hello World\n", 13 Hello World) = 13
exit_group(0)                           = ?
+++ exited with 0 +++

ganton@mint-virtual:~/SdeC/Practico_4/Sistemas-de-computacion/part1$
```

Figura 16: Comando strace.

Se pueden ver varias llamadas al sistema, como por ejemplo `execve`, que se ejecuta al principio y sirve para cargar el archivo ejecutable en la memoria e iniciar el proceso. La llamada al sistema más interesante es `write`, que es la que realmente se encarga de imprimir los caracteres.

7. ¿Qué es un segmentation fault? ¿Cómo lo maneja el kernel y cómo lo hace un programa?

Respuesta 7: Un segmentation fault ocurre cuando un programa intenta acceder a una zona de la RAM para la cual no tiene permiso, o cuando intenta hacer una operación en una zona prohibida (como intentar escribir en una parte de memoria configurada como “solo lectura”).

Quando ocurre un segmentation fault, el procesador pausa el programa y lanza una interrupción (excepción) que le devuelve el control al kernel; quien identifica qué proceso cometió la falta y envía una señal SIGSEGV (Segmentation Violation). A su vez, la acción por defecto es terminar el proceso y volcar el estado de la memoria del programa en un archivo core (core dump) para poder realizar un análisis del fallo.

Los programas pueden utilizar llamadas al sistema como `sigaction()` para que se ejecuten cuando detecten una señal `SIGSEGV` en lugar de terminar el proceso. Este tipo de funciones se utilizan para guardar registros de error, cerrar archivos abiertos o liberar recursos críticos antes de terminar el proceso, ya que casi nunca se logra que el programa siga funcionando normalmente.

8. Intentar firmar un módulo de kernel y documentar el proceso. <https://askubuntu.com/questions/770205/hot-to-sign-kernel-modules-with-sign-file>

Respuesta 8: Para hacer este punto fue necesario crear otra máquina virtual para activar la configuración de UEFI y de Secure Boot, y por ello se ve un nombre distinto en consola.

Primero es necesario crear las claves privada y pública, mediante OpenSSL:

```
openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes  
-days 365 -subj "CN=Clave_TP4_Sistemas_de_Computacion/"
```

MOK.priv es la clave privada, que es la que se utiliza para firmar el módulo.

MOK.der es la clave pública, que es la que se utiliza para verificar la firma privada.

rsa:2048 indica que se está usando una encriptación RSA de 2048 bits.

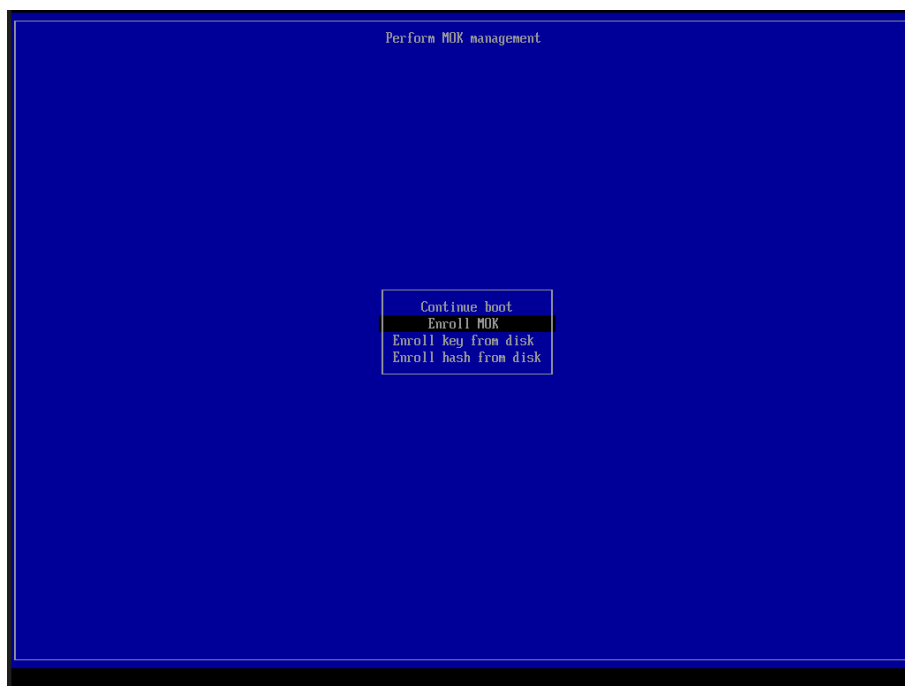


Figura 20: Enroll MOK.

Se selecciona Continue:



Figura 21: Continue.

Después se confirma que se quiere agregar la clave:

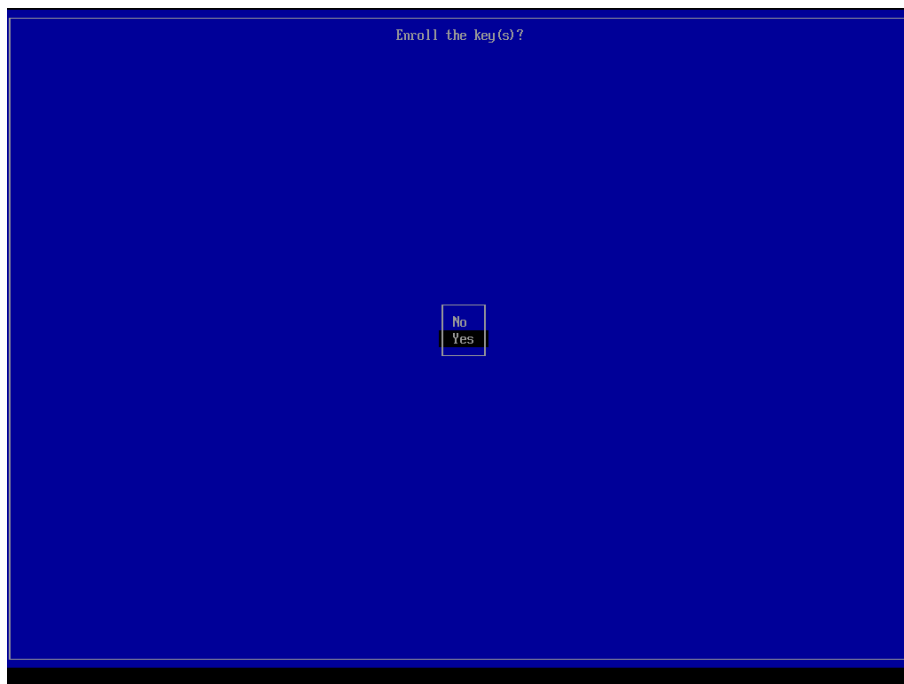


Figura 22: Confirmación.

Por último, se introduce la clave que se creó en la figura 18 y se reinicia el equipo:

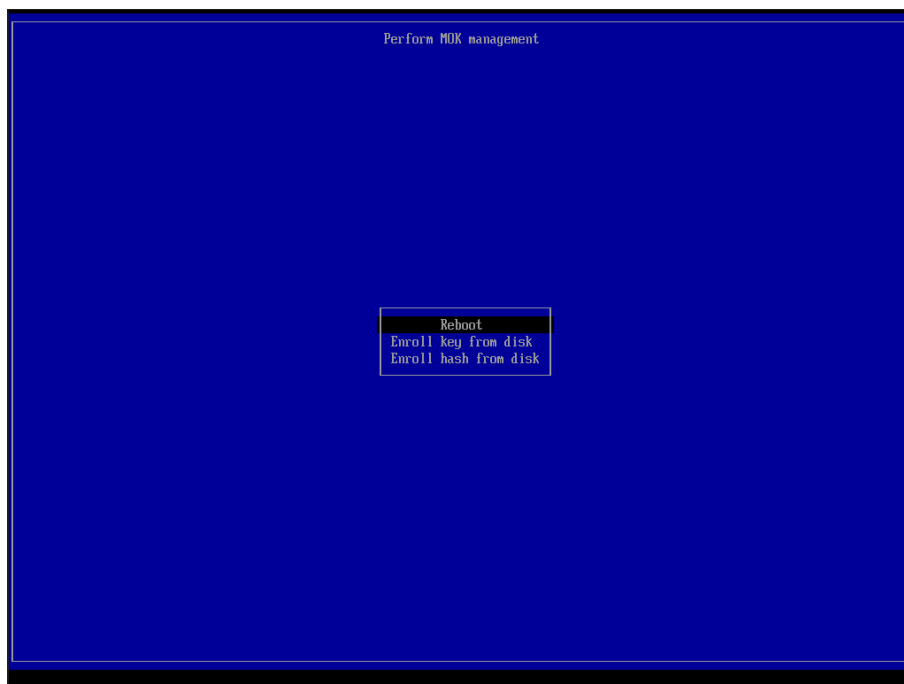


Figura 23: Reinicio luego de introducir la clave.

Ahora hay que firmar el módulo utilizando un script de firma que ya viene con el kernel:

```
/usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der mi_modulo.ko
```

```
Terminal - gaston@TP4-VirtualBox: ~/SdeC/TP4
File Edit View Terminal Tabs Help
gaston@TP4-VirtualBox:~/SdeC/TP4$ /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der mimodulo.ko
gaston@TP4-VirtualBox:~/SdeC/TP4$
```

Figura 24: Firma del Módulo.

Ahora se verifica que se firmó correctamente y se carga el módulo:

```
Terminal - gaston@TP4-VirtualBox: ~/SdeC/TP4
File Edit View Terminal Tabs Help
gaston@TP4-VirtualBox:~/SdeC/TP4$ modinfo mimodulo.ko | grep -E 'signer|sig_key|sig_hashalgo'
signer:      SistemasDeComputacionTP4
sig_key:     1A:9B:B3:31:83:CB:C5:C0:43:2C:F8:D3:82:64:20:D3:72:5A:63:3E
sig_hashalgo: sha256
gaston@TP4-VirtualBox:~/SdeC/TP4$ sudo insmod mimodulo.ko
[sudo] password for gaston:
gaston@TP4-VirtualBox:~/SdeC/TP4$
```

Figura 25: Verificación de la firma y carga del módulo.

Se verifica que se cargó correctamente el módulo:

```
Terminal - gaston@TP4-VirtualBox: ~/SdeC/TP4
File Edit View Terminal Tabs Help
tWebEngineProcess" pid=481 comm="apparmor_parser"
5.288096] audit: type=1400 audit(1780288177.555:8): apparmor="STATUS" operation="profile_load" profile="unconfined" name="b
uildah" pid=485 comm="apparmor_parser"
5.289052] audit: type=1400 audit(1780288177.556:9): apparmor="STATUS" operation="profile_load" profile="unconfined" name="b
alena-etcher" pid=483 comm="apparmor_parser"
5.292212] audit: type=1400 audit(1780288177.558:10): apparmor="STATUS" operation="profile_load" profile="unconfined" name="
cam" pid=488 comm="apparmor_parser"
5.329015] intel rapl msr: PL4 support detected.
5.722419] snd intel8x0 0000:00:05.0: allow list rate for 1028:0177 is 48000
5.962655] kauditd printk skb: 116 callbacks suppressed
5.962658] audit: type=1400 audit(1780288178.228:127): apparmor="STATUS" operation="profile_replace" info="same as current p
rofile, skipping" profile="unconfined" name="rsyslogd" pid=667 comm="apparmor_parser"
6.324368] NET: Registered PF_QIPCRTR protocol family
7.056482] audit: type=1400 audit(1780288179.323:128): apparmor="DENIED" operation="capable" class="cap" profile="/usr/sbin/
cupsd" pid=822 comm="cupsd" capability=12 capname="net_admin"
7.591185] Lockdown: Xorg: raw io port access is restricted; see man kernel lockdown.7
9.015327] systemd-journald[308]: /var/log/journal/683ccf2c604f4da6b2a7a92d408ac5e8/user-1000.journal: Journal file uses a d
ifferent sequence number ID, rotating.
9.021643] e1000: enp0s3 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
19.248114] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1851694329 wd_nsec: 1851693444
25.776677] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 5903816577 wd_nsec: 5903813850
589.044941] mimodulo: loading out-of-tree module taints kernel.
589.049062] Modulo cargado en el kernel.
gaston@TP4-VirtualBox:~/SdeC/TP4$
```

Figura 26: Verificación del Módulo cargado.

9. Agregar evidencia de la compilación, carga y descarga de su propio módulo imprimiendo el nombre del equipo en los registros del kernel.

Respuesta 9: Para lograr imprimir el nombre del equipo es necesario cambiar la línea del código en C donde se imprime “Modulo cargado en el kernel” (mediante printk), por lo que es necesario compilar de nuevo con Makefile y volver a firmar. Cabe destacar que no es necesario volver a hacer el proceso con MOK porque allí se registró una clave pública, y no un módulo, entonces se puede volver a utilizar la misma clave para firmar este nuevo módulo ligeramente modificado.

Una vez firmado el módulo modificado, es necesario remover el viejo y cargar el nuevo:


```

Terminal - gaston@TP4-VirtualBox: ~/SdeC/TP4
File Edit View Terminal Tabs Help

The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
CC [M] mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
  8 | int modulo_lin_init(void)
    | ^~~~~~
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
 17 | void modulo_lin_clean(void)
    | ^~~~~~
MODPOST Module.symvers
CC [M] mimodulo.mod.o
LD [M] mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/gaston/SdeC/TP4'
make[1]: Leaving directory '/usr/src/linux-headers-6.14.0-37-generic'
gaston@TP4-VirtualBox:~/SdeC/TP4$ /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der mimodulo.ko
gaston@TP4-VirtualBox:~/SdeC/TP4$ modinfo mimodulo.ko | grep -E 'signer|sig_key|sig_hashalgo'
signer:      SistemasDeComputacionTP4
sig_key:     1A:9B:B3:31:83:CB:C5:C0:43:2C:F8:D3:82:64:20:D3:72:5A:63:3E
sig_hashalgo: sha256
gaston@TP4-VirtualBox:~/SdeC/TP4$ sudo rmmod mimodulo.ko
gaston@TP4-VirtualBox:~/SdeC/TP4$ sudo insmod mimodulo.ko
gaston@TP4-VirtualBox:~/SdeC/TP4$
    
```

Figura 27: Carga del Módulo Modificado.

Por último, se verifica mediante el comando `dmesg` que el módulo modificado imprime un mensaje personalizado:

```

Terminal - gaston@TP4-VirtualBox: ~/SdeC/TP4
File Edit View Terminal Tabs Help

[11.289052] audit: type=1400 audit(1780288177.556:9): apparmor="STATUS" operation="profile_load" profile="unconfined" name="b
alena-etcher" pid=483 comm="apparmor_parser"
[5.292212] audit: type=1400 audit(1780288177.558:10): apparmor="STATUS" operation="profile_load" profile="unconfined" name="
cam" pid=488 comm="apparmor_parser"
[5.329015] intel_rapl_msr: PL4 support detected.
[5.722419] snd_intel8x0 0000:00:05:0: allow list rate for 1028:0177 is 48000
[5.962655] kauditd_printk_skb: 116 callbacks suppressed
[5.962658] audit: type=1400 audit(1780288178.228:127): apparmor="STATUS" operation="profile_replace" info="same as current p
profile, skipping" profile="unconfined" name="rsyslogd" pid=667 comm="apparmor_parser"
[6.324368] NET: Registered PF_QIPCRTR protocol family
[7.056482] audit: type=1400 audit(1780288179.323:128): apparmor="DENIED" operation="capable" class="cap" profile="/usr/sbin/
cupsd" pid=822 comm="cupsd" capability=12 capname="net_admin"
[7.591185] Lockdown: Xorg: raw io port access is restricted; see man kernel_lockdown.7
[9.015327] systemd-journal[308]: /var/log/journal/683ccf2c604f4da6b2a7a92d408ac5e8/user-1000.journal: Journal file uses a d
ifferent sequence number ID, rotating.
[9.021643] e1000: enp0s3 NIC Link is Up 1000 Mbps Full Duplex, Flow Control: RX
[19.248114] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1851694329 wd_nsec: 1851693444
[25.776677] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 5903816577 wd_nsec: 5903813850
[589.044941] mimodulo: loading out-of-tree module taints kernel.
[589.049062] Módulo cargado en el kernel.
[1323.675436] Módulo descargado del kernel.
[1334.448672] Módulo cargado en el kernel por Gaston Yamaha.
gaston@TP4-VirtualBox:~/SdeC/TP4$
    
```

Figura 28: Mensaje personalizado.

10. ¿Qué pasa si un compañero con secure boot habilitado intenta cargar un módulo firmado por mí?

Respuesta 10: En este caso el módulo no se cargaría porque el Secure Boot verificaría que la firma no se encuentra en su lista de confianza. Una posible solución sería compartir la clave pública para que el compañero la cargue mediante MOK (Machine Owner Key) en la lista de confianza, entonces ahora sí se podría cargar el módulo.

11. Dada la siguiente nota: <https://arstechnica.com/security/2024/08/a-patch-microsoft-spent-2-years-preparing-is-making-a-mess-for-some-linux-users/>

a. ¿Cuál fue la consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual (Linux y Windows)?

Respuesta a: La consecuencia principal fue que los sistemas con arranque dual no podían arrancar el equipo con Linux, y en su lugar se imprimía en pantalla un mensaje de error. Una de las soluciones era borrar los cambios introducidos por la última actualización, mientras que la otra (menos recomendable) era desactivar Secure Boot por el momento.

b. ¿Qué implicancia tiene desactivar Secure Boot como solución al problema descrito en el artículo?

Respuesta b: Desactivar Secure Boot implica que el equipo quede vulnerable a malware de muy bajo nivel, como los Rootkits de UEFI, de forma que el equipo puede llegar a ejecutar un módulo de kernel sin firmar, accediendo al nivel de privilegio más alto y controlando el hardware y software.

c. ¿Cuál es el propósito principal del Secure Boot en el proceso de arranque de un sistema?

Respuesta c: El propósito principal es asegurarse de que cada componente que participa en el inicio del pc tenga una firma digital válida, con el objetivo de aumentar la seguridad en el arranque.

5. Link al Repositorio

<https://github.com/GastonYomaha/SistemasDeComputacion/tree/main/TP4>